04-Observability 보충 자료

A2A 샘플로 보는 에이전트 관찰성

Incident Response 샘플 코드로 확인하는 트레이스 · 로그 · 지표 · 평가
Building Advanced Agentic Systems on AWS

트레이스
ADOT · X-Ray · Transaction Search

로그
Vended Logs · 로그 상관관계

지표 · 경보
bedrock-agentcore 네임스페이스

평가
AgentCore Evaluations

이 자료의 구성

A

교안 ↔ 코드 매핑

교안 슬라이드를 진행하며 화면에 띄울 코드 위치 15곳, 성숙도 진단, 요청 하나의 스팬 구조

B

추가적인 기술 내용

ADOT 설치, 트레이스 전달 리소스, 스팬 규약, A2A 트레이스 전파, 이중 내보내기, 루프 · 비용, 로그 상관 관계, 평가, 경보

C

코드에서 찾은 함정 · 개선 과제

배포를 막는 버그를 포함한 6가지와 개선 코드

관찰성 지도: 무엇이 어디로 가는가

교안 #7, #25

	Host 런타임 Google ADK	Monitoring 런타임 Strands Agents	WebSearch 런타임 OpenAI Agents SDK	Gateway · Memory AgentCore 서비스
자동 계측	ADOT ✓	ADOT ✓	ADOT ✓	서비스 자체
프레임워크 스팬	ADK 내장 OTel	Strands 내장 OTel	openinference	—
APPLICATION_LOGS	✓ · 30일 *	✓ · 보존 무기한	✓ · 30일 *	Gateway ✓ · Memory X
TRACES → X-Ray	✓ 커스텀 Lambda	? 중복 논리 ID	✓ 커스텀 Lambda	Gateway ✓ · Memory X
트레이스 외부 전송	—	—	OpenAI 플랫폼에도	—

공통 상관 키 프런트엔드 Session-Id 헤더 → Host context.session_id → 원격 런타임 2개에 그대로 전달 (agent.py:49)

* 이 zip 스냅샷에서는 RetentionInDays 들여쓰기 오류로 템플릿 자체가 파싱되지 않음 → 파트 B

A

교안 ↔ 코드 매핑

교안을 진행하면서 화면에 띄울 코드 위치와 관찰성 진단

모니터링 아키텍처: 꼭 짚을 코드 8곳

교안 #4–18

교안	개념	코드 위치	짚을 포인트
#8	세션 · 트레이스 · 스팬	`frontend/.../chatService.ts:45`	Session-Id 헤더 하나가 세 런타임의 session.id
#9	추론 트레이스	`host_adk_agent/main.py:72`	runner.run_async — transfer_to_agent 판단이 도구 스팬으로
#11	세션 지표	`(Runtime 자동 발행)`	bedrock-agentcore 네임스페이스 — 코드 0줄
#12	트레이스 지표 · 토큰	`monitoring_strands_agent/agent.py:47`	stream_async — Strands 스팬에 토큰 사용량 속성
#13	스팬 · ADOT	`*/Dockerfile CMD (42, 47)`	opentelemetry-instrument로 프레임워크 스팬 수집
#14	도구 실패 · 저하	`host_adk_agent/agent.py:54`	timeout 300초 · 재시도 없음 → 하위 실패는 그대로 오류
#16	서비스별 관찰성	`monitoring_agent.yaml:1693`	Gateway 로그 전달 ✓ · Memory 리소스 2개는 전달 없음
#18	다중 에이전트 가시성	`web_search_openai_agents/main.py:14`	OpenAIAgentsInstrumentor — 세 프레임워크, 세 스팬 규약

관찰성 통합 · 평가: 꼭 짚을 코드 7곳

교안 #21–49

교안	개념	코드 위치	짚을 포인트
#21	ADOT 설치	`*/Dockerfile:23-24`	aws-opentelemetry-distro — 따옴표가 없어 버전 미고정
#25	런타임 자동 계측	`*_agent.yaml EnvironmentVariables`	OTEL_* 환경변수 없음 — 런타임이 주입
#25	Transaction Search	(없음)	계정 단위 설정 — 샘플은 활성화되어 있다고 가정
#26	내보내기 대상	`web_search_openai_agents/main.py:14`	OpenAI SDK 기본 프로세서가 OpenAI 플랫폼으로도 전송
#28	루프 방지	`host main.py:72 · *_agent.yaml Runtime`	RunConfig · LifecycleConfiguration 미지정 → 기본값
#29	경보 구성	(없음)	경보 · 대시보드 · 지표 필터 없음
#37–48	Evaluations	(없음)	평가기 · 온라인 평가 미구성 → 파트 B에서 설계

성숙도 진단: 수집은 됐고, 분석은 아직

교안 #4, #6

4단계
사전 예방적 · 자동 근본 원인

3단계
상관관계 · 이상 탐지

2단계
원격 측정 분석 · 인사이트

1단계
원격 측정 데이터 수집

▲ 현재 샘플 · 다음 목표는 2단계

트레이스 — 부분

세 런타임 자동 계측 ✓ · Monitoring 전달 설정 덮어쓰기 · A2A 전파 미확인

로그 — 부분

APPLICATION_LOGS 전달 ✓ · 트레이스 ID 없는 평문 로그, 보존 기간 불일치

지표 — 부분

런타임 기본 지표 자동 ✓ · 사용자 지정 지표 · 지표 필터 없음

경보 · 대시보드 — 빈틈

경보 0개 · 대시보드 0개 → 루프 · 비용 이상을 아무도 모름

평가 — 빈틈

평가기 · 온라인 평가 없음 · 라우팅 품질을 측정할 방법 없음

비용 · 샘플링 — 빈틈

샘플러 · 보존 계층 미설정 · 토큰 속성 규약이 프레임워크마다 다름

요청 하나를 펼치면: 세션 · 트레이스 · 스팬

교안 #8, #11–13



개념도 — 스팬 이름은 프레임워크 버전에 따라 다름. 세로 줄 = 스팬, 가로 길이 = 상대 시간

AgentCore 서비스별 관찰성: 샘플이 켠 것과 끈 것

교안 #16–17

서비스	샘플 설정	판정
Runtime ×3	APPLICATION_LOGS (CFN) + TRACES (Lambda)	✓ · Monitoring은 ?
Gateway	APPLICATION_LOGS · TRACES	✓
Memory ×2	전달 설정 없음	추출 · 통합 관찰 불가
Identity	전달 설정 없음	토큰 교환 실패 추적 불가
Policy · Evaluations	사용 안 함	—

```
# 개선 예 — Memory 로그 전달
MemoryLogSource:
  Type: AWS::Logs::DeliverySource
  Properties:
    Name: !Sub ${AgentMemory}-logs
    LogType: APPLICATION_LOGS
    ResourceArn:
      !GetAtt AgentMemory.MemoryArn

MemoryLogDelivery:
  Type: AWS::Logs::Delivery
  Properties:
    DeliverySourceName: ...
    DeliveryDestinationArn: ...
# 지원 LogType은 문서로 확인
```

눈여겨볼 점 Memory는 커스텀 추출 전략(MonitoringIssueTracker)이 별도 모델을 호출하는데, 그 실패와 지연을 볼 곳이 없습니다

B

추가적인 기술 내용

계측 설치부터 트레이스 전달, 스팬 규약, 전파, 경보와 평가까지

세 프레임워크, 세 가지 스팬 규약

NEW · 교안 #18

	Host · ADK	Monitoring · Strands	WebSearch · OpenAI SDK
계측 출처	ADK 내장 OTel 트레이서	Strands 내장 OTel 트레이서	openinference 계측기 (main.py:14)
규약	GenAI 시맨틱 규약 (gen_ai.*)	GenAI 시맨틱 규약 (gen_ai.*)	OpenInference (llm.*, openinference.*)
토큰 속성	gen_ai.usage.input_tokens	gen_ai.usage.input_tokens	llm.token_count.prompt
에이전트 식별	agent 이름 = root_agent	Agent(name="Monitoring Agent")	Agent(name="WebSearch_Agent")
모델	Gemini (LiteLLM 경유 가능)	Bedrock Claude	OpenAI GPT

1 공통 속성 붙이기

세 실행기에서 session.id · enduser.id · agent.name을 같은 이름으로 스팬에 기록

2 Collector에서 변환

OTel Collector transform 프로세서로 llm.token_count.* → gen_ai.usage.*

3 프레임워크 줄이기

AWS 전환 시 Strands로 통일하면 규약도 하나로

트레이스가 A2A 경계를 넘는가

NEW · 교안 #8, #18

session.id

명시적으로 전달 ✓

agent.py:49 헤더에 직접 넣음

→ 세션 단위 상관관계 보장

traceparent

자동 계측에 의존 ?

코드에는 없음 · httpx 계측이

주입하면 부모-자식으로 연결

- **확인 방법** Transaction Search에서 Host 트레이스를 열어 Monitoring 스팬이 자식으로 보이는지
- **안 이어지면** 같은 session.id를 가진 별개 트레이스 3개 → 지연의 원인 구간을 한 화면에서 못 봄

에이전트끼리 호출할 때, 같은 세션 ID를 전달하는 것만으로 실행 과정이 하나의 트레이스로 연결되지는 않는다

```python
# host_adk_agent/agent.py — _create_client 개선 예
from opentelemetry.propagate import inject

async def _inject_trace(request: httpx.Request):
    inject(request.headers)   # traceparent · baggage

client = httpx.AsyncClient(
    headers={
        "Authorization": f"Bearer {bearer_token}",
        "X-Amzn-Bedrock-AgentCore-Runtime-Session-Id":
            session_id,
        ...},
   event_hooks={"request": [_inject_trace]},
    timeout=httpx.Timeout(timeout=300.0))

# 헤더를 생성 시점에 고정하지 말 것:
# 트레이스 컨텍스트는 요청마다 다르다
# 요청을 보낼 때마다 _inject_trace 함수로 추적 정보를 전달
```

트레이스가 A2A 경계를 넘는가

NEW · 교안 #8, #18

- session.id와 traceparent는 서로 다른 역할

구분	`session.id`	`traceparent`
의미	어떤 대화·세션에 속하는가?	어떤 실행 흐름에서 이어진 호출인가?
비유	고객 상담 접수 번호	담당자 사이의 업무 인계 기록
확인할 수 있는 것	같은 세션에 속한 작업들	누가 누구를 호출했으며 어디에서 시간이 걸렸는지
적용 범위	한 세션에 여러 요청이 포함될 수 있음	하나의 실행 흐름을 여러 서비스에 걸쳐 연결

WebSearch의 트레이스는 두 곳으로 간다

NEW · 교안 #26, #31



- **트레이스에 담기는 것** 프롬프트, 도구 입력 · 결과 (Tavily 결과, 메모리 내용)
- **AWS 전환 후** 키가 없어 내보내기 오류 로그가 계속 쌓임

```
# web_search_openai_agents/main.py:7, 14 — 현재
from openinference.instrumentation.openai_agents \
    import OpenAIAgentsInstrumentor

OpenAIAgentsInstrumentor().instrument()
# 기본 OpenAI 트레이스 프로세서도 그대로 동작
```

```
# 개선 — openinference 프로세서만 남기기
OpenAIAgentsInstrumentor().instrument(
    exclusive_processor=True)   # 버전 문서로 확인

# 또는 SDK API로 기본 프로세서 교체
from agents import set_trace_processors
set_trace_processors([])   # 이후 계측기 등록
# set_tracing_disabled(True)는 계측도 끈다
```

보고서를 CloudWatch라는 새 보관함에 넣도록 설정했지만, 기존 OpenAI 보관함으로 보내는 기능도 남아 있는 상황입니다.

루프와 비용: 샘플의 안전장치 점검

NEW · 교안 #28–30

안전장치	샘플 상태	판정
런타임 수명 주기	LifecycleConfiguration 없음 → 유휴 15분 · 최대 8시간	기본값
Host LLM 호출 상한	RunConfig 없음 → max_llm_calls 500 (main.py:72)	기본값
Strands 반복 상한	명시적 상한 없음 (agent.py:35)	없음
OpenAI 턴 상한	max_turns 기본 10 (agent.py:52)	기본값
A2A 호출	timeout 300초 · 재시도 · 차단기 없음	부분
경보 · 샘플러	경보 0개 · OTEL_TRACES_SAMPLER 미지정	없음

```
# monitoring_strands_agent — 개선 예
from strands.hooks import (
    HookProvider, BeforeToolCallEvent)

class ToolBudget(HookProvider):
    def __init__(self, limit=15):
        self.limit, self.n = limit, 0
    def register_hooks(self, reg):
        reg.add_callback(
            BeforeToolCallEvent, self.check)
    def check(self, event):
        self.n += 1
       if self.n > self.limit:
           event.cancel_tool = (
               "도구 호출 한도 초과")

# Agent(hooks=[monitoring_hooks,
#              ToolBudget()])
# 턴마다 n 초기화 · Strands 1.16 기준
```

로그는 트레이스와 엮이는가

NEW · 교안 #4, #30

```
# 현재 — */main.py
logging.basicConfig(level=logging.INFO)

# CloudWatch에 남는 모양
INFO:agent_executor:Executing task 7f3c...
INFO:memory_hook:Retrieved 3 memories
#   ↑ 어느 세션 · 어느 트레이스인지 알 수 없음
```

```
# 개선 — 런타임 환경변수 (*_agent.yaml)
EnvironmentVariables:
  OTEL_PYTHON_LOG_CORRELATION: "true"

# 결과
INFO [trace_id=160fc209… span_id=81346de8…]
     agent_executor: Executing task 7f3c...
```

항목	현재	개선
상관 키	없음 (평문 로그)	trace_id · span_id 자동 삽입 + session_id를 extra로
보존 기간	Host · WebSearch 30일 (깨진 YAML) · Monitoring 무기한	세 로그 그룹 동일하게 · 교안 #30 계층형 보존
로그 양	memory_hook 로그 5곳 · 전체 텍스트 기록	INFO는 요약, 원문은 DEBUG

로그에 추적 ID를 붙여서, 어떤 요청을 처리하다 남긴 메시지인지 알 수 있게 하자

Evaluations를 샘플에 붙인다면

NEW · 교안 #39–45

대상	수준	평가기
Host	세션	목표 성공률
Host	스팬 · transfer_to_agent	**도구 선택 정확성 = 라우팅 정확도**
Monitoring	스팬 · FilterLogEvents	파라미터 선택 정확성
WebSearch	트레이스	충실도 · 관련성
Host 최종 답	트레이스	**사용자 지정: 근거 인용**

- **라우팅 = 도구 선택** ADK의 위임은 transfer_to_agent 도구 호출이므로 내장 평가기로 측정 가능
- **먼저 확인** 평가기가 읽는 스팬 규약 — openinference 스팬 해석 여부

```
# 교안 #43 형식 – 도메인 평가기
evaluator = client.create_evaluator(
  name="incident-evidence-citation",
  evaluation_instructions=
    "Check whether {agent_response} cites"
    " metric values, units, timestamps and"
    " resource IDs found in {context}.",
  evaluator_model="<Bedrock 모델 ID>",
  scoring_schema={
    "0.0": {"label": "No evidence"},
    "0.5": {"label": "Partial"},
    "1.0": {"label": "Fully grounded"}},
  evaluation_level="Trace")
```

온라인 평가는 샘플링 비율(예: 10%)로 시작하고, 점수 하락을 다음 슬라이드의 계층 3 경보로 연결

에이전트가 실행되는지 확인하는 데서 나아가, 일을 제대로 했는지 점수로 평가하자

경보 3계층을 샘플에 대입하면

NEW · 교안 #27–29

계층 1 · 자동 응답

- 도구 예산 훅으로 루프 차단
- A2A 재시도 · 회로 차단기 (현재 없음)
- Gateway 토큰 재발급 (MCPClient 고정)

계층 2 · 기술 팀

- 런타임 오류율 · 지연 시간 · 스로틀
- 'Error executing task' 지표 필터 → A2A 하위 실패
- 세션 지속 30분 초과 (교안 #29)

계층 3 · 비즈니스

- 라우팅 정확도 점수 하락
- 근거 인용 점수 하락
- 세션당 토큰 기준선 2배 초과

```
# 계층 2 — A2A 하위 실패 지표 필터 (agent_executor.py:177 · :156의 로그 문구)
A2ATaskErrorFilter:
  Type: AWS::Logs::MetricFilter
  Properties:
    LogGroupName: !Ref AgentRuntimeLogGroup
    FilterPattern: '"Error executing task"'
    MetricTransformations: [{MetricNamespace: IncidentAgents, MetricName: A2ATaskErrors, MetricValue: "1"}]
```

C

코드에서 찾은 함정 · 개선 과제

배포를 막는 버그부터 보이지 않는 데이터 유출까지 6가지와 개선 코드

코드에서 찾은 함정: 토론 · 실습 과제

교안 #16–48

1 깨진 YAML

교안 #25

RetentionInDays 들여쓰기 오류 → Host · WebSearch 템플릿 파싱 실패

`host_agent.yaml:736 · web_search:919`

2 중복 논리 ID

교안 #25

XRayTracesSetup* 3개를 두 번 정의 → 런타임 TRACES 전달 누락 가능

`monitoring_agent.yaml:965, 1465`

3 ADOT 버전 미고정

교안 #21

>=가 셸 리다이렉트로 해석 → 매 빌드 최신 버전

`*/Dockerfile:23`

4 전달 리소스 잔존

교안 #25

delete_delivery에 source 이름을 넘김 → 삭제 실패를 경고로 삼킴

`*.yaml delete_delivery(id=…)`

5 트레이스 이중 전송

교안 #26

OpenAI SDK 기본 프로세서가 OpenAI 플랫폼으로도 전송

`web_search_openai_agents/main.py:14`

6 판단 장치 부재

교안 #29, #37

경보 · 대시보드 · 평가 0개, Memory · Identity 로그 전달 없음

`cloudformation/*.yaml`

개선 과제: 몇 줄로 고치는 관찰성

교안 #21–30

1 템플릿 고치고 검증하기

함정 1 · 2

```
# host_agent.yaml · web_search_agent.yaml
    Properties:
     RetentionInDays: 30
     LogGroupClass: STANDARD
# 배포 전 검증 — 중복 키도 잡아냄
cfn-lint cloudformation/*.yaml
```

2 ADOT 버전 고정

함정 3

```
# */Dockerfile
RUN uv pip install aws-opentelemetry-distro>=0.10.1
RUN uv pip install "aws-opentelemetry-distro==<검증 버전>"
# 확인
docker run --rm <image> pip show aws-opentelemetry-distro
```

3 A2A 트레이스 이어 붙이기

교안 #18

```
# host_adk_agent/agent.py _create_client
from opentelemetry.propagate import inject
async def _inject_trace(req): inject(req.headers)
httpx.AsyncClient(headers=headers,
   event_hooks={"request": [_inject_trace]}, ...)
```

4 로그에 트레이스 ID

교안 #4

```
# *_agent.yaml AgentRuntime.EnvironmentVariables
OTEL_PYTHON_LOG_CORRELATION: "true"
# Logs Insights
fields @timestamp, @message
| filter @message like /trace_id=160fc209/
```

정리

수집에서 판단까지 이어져야 관찰성이다

수집

런타임이 자동으로 수집한다. 하지만 Dockerfile 한 줄과 YAML 한 칸이 수집을 조용히 끈다

연결

session.id는 이미 하나다. traceparent와 스팬 규약까지 맞춰야 세 에이전트가 한 화면에 보인다

판단

경보로 알리고 평가로 매긴다. 라우팅은 도구 선택이므로 내장 평가기로 측정할 수 있다